Simple Cipher Library - Student Guide

Overview

This library implements a **simple asymmetric cipher** for teaching encryption concepts in network programming. It uses modular arithmetic with multiplicative inverses to create key pairs that enable bidirectional encrypted communication between a client and server.

⚠ **Important:** This cipher is for **educational purposes only**. It is not cryptographically secure and should never be used in production systems.

What is Asymmetric Encryption?

In traditional **symmetric encryption** (like AES), the same key is used to both encrypt and decrypt: - Alice encrypts with Key K → Bob decrypts with the same Key K

In **asymmetric encryption** (like RSA), different but mathematically related keys are used: - Alice encrypts with Key A → Bob decrypts with Key B - Bob encrypts with Key B → Alice decrypts with Key A

Why use asymmetric encryption? - Keys can be different, but still work together - Each party can have their own encryption/decryption capabilities - Supports bidirectional communication without sharing the same secret

---

The Multiplicative Cipher (Our Approach)

Our cipher is based on **modular multiplicative inverses** using modulus 64.

Core Mathematics

Encryption Formula:

`C = (P × e) mod 64`

Decryption Formula:

`P = (C × d) mod 64`

Where: - `P` = plaintext byte (value 0-63) - `C` = ciphertext byte (value 0-63) - `e` = encryption key - `d` = decryption key - **Key Relationship:** `(e × d) mod 64 = 1`

The keys `e` and `d` are **modular multiplicative inverses** of each other.

Why Does This Work?

The magic is in the mathematical property:

```
Decrypt(Encrypt(P)) = ((P × e) × d) mod 64
                    = (P × (e × d)) mod 64
                    = (P × 1) mod 64
                    = P
```

Because `(e × d) mod 64 = 1`, multiplying by both keys returns the original value!

Example Calculation

Let's encrypt and decrypt `P = 10` using keys `e = 7, d = 55`:

Encryption:

`C = (10 × 7) mod 64 = 70 mod 64 = 6`

Decryption:

`P = (6 × 55) mod 64 = 330 mod 64 = 10`

We got our original value back!

Valid Keys

Why Modulus 64?

We chose 64 because: 1. **64 = 2^6** (a clean power of 2) 2. Any **odd number** from 1-63 is coprime with 64 3. This gives us **32 valid keys** to choose from 4. The math is simple enough to verify by hand

Valid Keys = All Odd Numbers

Valid keys (32 total):

```
1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31,
33, 35, 37, 39, 41, 43, 45, 47, 49, 51, 53, 55, 57, 59, 61, 63
```

Invalid keys:

```
Even numbers: 2, 4, 6, 8, ..., 62
(They share the factor 2 with 64, so no inverse exists)
```

Finding the Inverse Key

To find the decryption key d for encryption key e, we search:

```
for (d = 1; d < 64; d++) {
    if ((e × d) mod 64 == 1) {
        return d;  // Found it!
    }
}
```

Example: If e = 7:

```
7 × 1 = 7 mod 64 = 7     □
7 × 3 = 21 mod 64 = 21   □
7 × 9 = 63 mod 64 = 63   □
...
7 × 55 = 385 mod 64 = 1  □  Found it!
```

Result: For key e = 7, the inverse is d = 55

The Character Alphabet

The cipher maps byte values 0-63 to printable ASCII characters:

```
Index  0–25:  A–Z  (uppercase letters)
Index 26–51:  a–z  (lowercase letters)
Index 52–61:  0–9  (digits)
Index    62:  ' '  (space)
Index    63:  ','  (comma)
```

Full 64-character alphabet:

```
ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789 ,
```

Why only 64 characters? - Matches our modulus (0-63 range) - Covers most common text - Simple to implement and debug

What about other punctuation? - Characters like **.**, !, ?, **:** are **not supported** - Only use the 64 characters listed above in your messages

---

Bidirectional Communication (The Hybrid Key Approach)

The Challenge: In a client-server system, both parties need to send and receive encrypted messages. Each party needs: - An encryption key (to encrypt outgoing messages) - A decryption key (to decrypt incoming messages)

The Solution: Generate TWO key pairs and distribute them strategically!

How It Works

Step 1: Server Generates Two Key Pairs

```
Keypair 1:  e1 = 7,  d1 = 55   (for Client → Server communication)
Keypair 2:  e2 = 11, d2 = 35   (for Server → Client communication)
```

Step 2: Create Hybrid Keys Each hybrid key contains both an encryption key and a decryption key:

```
Server's key = [d2, e1] = [35, 7]
  - Encrypt outgoing messages with d2 (35)
  - Decrypt incoming messages with e1 (7)

Client's key = [d1, e2] = [55, 11]
  - Encrypt outgoing messages with d1 (55)
  - Decrypt incoming messages with e2 (11)
```

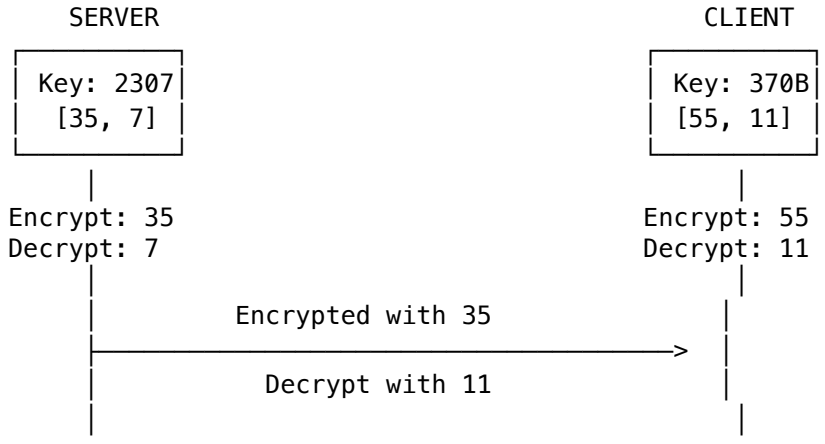
Step 3: The Magic Connection Notice how the keys interconnect:

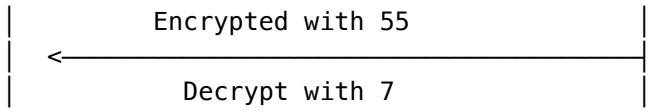
```
CLIENT ENCRYPTS WITH 55  →  [encrypted data]  →  SERVER DECRYPTS WITH 7
(using d1 from keypair 1)                        (using e1 from keypair 1)
                                                  Works! (55 and 7 are inverses)

SERVER ENCRYPTS WITH 35  →  [encrypted data]  →  CLIENT DECRYPTS WITH 11
(using d2 from keypair 2)                        (using e2 from keypair 2)
                                                  Works! (35 and 11 are inverses)
```

Why this works: - Client uses keypair 1's decryption key (d1) to encrypt - Server uses keypair 1's encryption key (e1) to decrypt - They're mathematical inverses, so it works! - Same logic for the reverse direction with keypair 2

Visual Representation





Key Storage Format

Each hybrid key is stored as a `uint16_t` (2 bytes):

```
Bits: [15 14 13 12 11 10  9  8 | 7  6  5  4  3  2  1  0]
      [Encryption Key (8 bits) | Decryption Key (8 bits)]
```

Example:

```
Server key = 0x2307 = [35, 7]
  Hex:     0x23          0x07
  Binary:  00100011      00000111
           ↑             ↑
           Encrypt (35)  Decrypt (7)
```

Extracting keys:

```c
crypto_key_t server_key = 0x2307;
uint8_t enc_key = GET_ENCRYPTION_KEY(server_key);  // Returns 35
uint8_t dec_key = GET_DECRYPTION_KEY(server_key);  // Returns 7
```

How This Applies to Your Assignment

Protocol Flow

1. **Client connects to server**
   - No encryption yet (no keys)
2. **Client sends key exchange request** (# command → MSG_KEY_EXCHANGE)

   `Client → Server: MSG_KEY_EXCHANGE request (no payload)`
3. **Server generates keys and responds**

   ```c
   crypto_key_t server_key, client_key;
   gen_key_pair(&server_key, &client_key);  // Server keeps server_key
   // Send client_key to the client
   memcpy(response->payload, &client_key, sizeof(crypto_key_t));
   ```
4. **Client receives and stores their key**

   ```c
   crypto_key_t session_key;
   memcpy(&session_key, response->payload, sizeof(crypto_key_t));
   // Now client can encrypt/decrypt!
   ```

5. **Encrypted communication begins**

```
Client: "!Hello" → Encrypt with session_key → Send MSG_ENCRYPTED_DATA
Server: Receive → Decrypt with server_key → Process → Encrypt → Send back
Client: Receive → Decrypt with session_key → Display
```

Using the Crypto Library Functions

For Key Exchange (Server Only)

```c
// Server generates keys when client requests
crypto_key_t server_key, client_key;
if (gen_key_pair(&server_key, &client_key) == RC_OK) {
    // Keep server_key for yourself
    // Send client_key to the client in MSG_KEY_EXCHANGE response
}
```

For Encrypting Messages (Client or Server)

```c
uint8_t plaintext[] = "Hello";
uint8_t encrypted[100];
int encrypted_len = encrypt_string(session_key, encrypted, plaintext, strlen(plaintex

if (encrypted_len > 0) {
    // Copy encrypted data to PDU payload
    memcpy(pdu->payload, encrypted, encrypted_len);
    pdu->header.payload_len = encrypted_len;
}
```

For Decrypting Messages (Client or Server)

```c
uint8_t decrypted[100];
int decrypted_len = decrypt_string(session_key, decrypted,
                                   received_pdu->payload,
                                   received_pdu->header.payload_len);

if (decrypted_len > 0) {
    decrypted[decrypted_len] = '\0';  // Add null terminator
    printf("Decrypted message: %s\n", decrypted);
}
```

For Debugging

```c
// Print PDU details (shows encrypted and decrypted data)
print_msg_info(pdu, session_key, CLIENT_MODE);  // or SERVER_MODE
```

Important Implementation Notes

String Conversion

The library handles ASCII ↔ byte index conversion automatically:

```
encrypt_string()  // Converts "Hello" → bytes → encrypts → returns encrypted bytes
decrypt_string()  // Decrypts bytes → converts to ASCII → returns string
```

You don't need to manually convert characters to indices!

Remember to Null-Terminate

```
int len = decrypt_string(key, output, input, input_len);
output[len] = '\0';  // IMPORTANT: decrypt_string doesn't add this!
```

Check for Invalid Keys

```
if (session_key == NULL_CRYPTO_KEY) {
    printf("Error: No encryption key established\n");
    // Can't encrypt/decrypt yet!
}
```

Character Restrictions

Only these characters work in encrypted messages: - Letters: A-Z, a-z - Digits: 0-9 - Space and comma: ' ',','

Invalid characters will cause `RC_INVALID_TEXT` error!

Security Limitations

⚠ **This cipher is NOT secure for real-world use:**

1. **Tiny keyspace**: Only 32 possible keys → Can try them all in milliseconds
2. **No randomness**: Same plaintext always produces same ciphertext
3. **Pattern leakage**: Repeated characters stay repeated when encrypted
4. **No authentication**: Can't verify who sent a message
5. **No integrity**: Can't detect if message was modified
6. **Malleable**: Attacker can predictably modify encrypted messages

Why teach it then? - Demonstrates asymmetric encryption concepts - Simple enough to understand completely - Shows key exchange and bidirectional communication - Easy to debug (can decrypt messages to verify) - Focus on network programming, not complex cryptography

---

Debugging Your Implementation

Common Issues

"No session key established" - Client hasn't requested key exchange yet - Send # command before trying `!message`

"Invalid character" error - Your message contains unsupported punctuation - Only use: `A-Za-z0-9 ,`

Messages decrypt to garbage - Using wrong key (client vs server key confusion) - Keys weren't exchanged properly - Check key values with `printf("key=0x%04x\n", key)`

Keys are 0xFFFF (NULL_CRYPTO_KEY) - Key exchange failed - Check `gen_key_pair()` return value - Verify server is sending correct key to client

Useful Debugging Commands

```
// Print the PDU details
print_msg_info(msg, session_key, CLIENT_MODE);

// Print key values
printf("Server key: 0x%04x (enc=%d, dec=%d)\n",
       server_key,
       GET_ENCRYPTION_KEY(server_key),
       GET_DECRYPTION_KEY(server_key));

// Test encryption/decryption
uint8_t test[] = "test";
uint8_t enc[10], dec[10];
encrypt_string(key, enc, test, 4);
decrypt_string(key, dec, enc, 4);
dec[4] = '\0';
printf("Original: %s, Decrypted: %s\n", test, dec);
```

Questions to Deepen Understanding

1. **Why are even numbers invalid as keys?**
   Answer
   Even numbers share the factor 2 with 64 (64 = 2^6). For a multiplicative inverse to exist, the key and modulus must be coprime (share no common factors except 1). Since even numbers and 64 both have factor 2, no inverse exists.
2. **What happens if we use the same key for both encryption and decryption?**

Answer
This only works for specific "self-inverse" keys where $k \times k \equiv 1 \pmod{64}$. For modulus 64, these are: 1, 63. Using k=1 doesn't encrypt (C=P), and k=63 creates a simple negation cipher. Neither is secure.

3. **Why do we need TWO key pairs instead of one?**
   Answer
   One key pair allows one-way communication. For bidirectional communication, we need each party to have different capabilities: Client uses keypair1 to encrypt, Server uses keypair2 to encrypt. This way both can send encrypted messages the other can decrypt.
4. **Could an attacker break this cipher?**
   Answer
   Yes, easily! With only 32 possible keys, trying all of them (brute force attack) takes milliseconds on any modern computer. Additionally, frequency analysis of encrypted messages would quickly reveal patterns. This is why it's educational only!
5. **How does this compare to RSA?**
   Answer
   RSA uses similar mathematical concepts (modular arithmetic, key pairs) but with MUCH larger numbers (2048+ bits vs our 6 bits). RSA's huge keyspace makes brute force impractical. RSA also uses prime numbers and more sophisticated math to prevent pattern analysis.
6. **What would we need to add for production use?**
   Answer
   - Much larger keyspace (2048+ bit keys)
   - Padding/randomization to prevent pattern analysis

   - Authentication (digital signatures)
   - Integrity checking (HMAC or authenticated encryption)
   - Secure key exchange protocol (like Diffie-Hellman)
   - Forward secrecy (new keys for each session)

   In practice, use established standards like TLS 1.3 with AES-GCM!

Further Reading

Want to learn more? Check out:

- **Modular Arithmetic**: The foundation of modern cryptography
- **RSA Algorithm**: Real asymmetric encryption used everywhere
- **Diffie-Hellman**: How to exchange keys over insecure channels
- **Extended Euclidean Algorithm**: Fast way to find modular inverses
- **AES-GCM**: Modern authenticated encryption (what you should actually use)
- **TLS/SSL**: How HTTPS encrypts web traffic

Summary

What you learned: - Asymmetric encryption uses different keys for encrypt/decrypt - Keys are mathematical inverses: $(e \times d) \bmod 64 = 1$ - Hybrid keys enable bidirectional communication - Key exchange happens before encrypted messages - Only specific characters can be encrypted (our 64-character alphabet)

What you're implementing: - Socket programming (TCP client/server) - Protocol design (PDU format, message types) - Key exchange handshake - Encrypted message handling

What the library handles for you: - Key generation (`gen_key_pair`) - String encryption (`encrypt_string`) - String decryption (`decrypt_string`) - Character conversion (ASCII $\leftrightarrow$ indices)

Focus on the networking and protocol - the crypto is done!

This cipher is provided for educational purposes. Feel free to experiment and learn!